Содержание

1. Введение 2
2. Зависимости 4
3.1. Интеграция с CMake 5
3.2. Интеграция с QMake 8
3.3 Упаковка скомпилированной библиотеки в .deb пакет 9
4. Класс `simpleapi::Config` 10
4.1. Формат JSON 14
4.2. Формат INI 16
4.3. Формат XML (TODO) 18
4.4. Формат YAML (TODO) 19
5. Работа с комментариями 20
6. Сетевое взаимодействие 25
6.1. Сокеты 26
6.2. Поток для автоматизации работы сокетов 28
6.3. Описание структуры пакетов SimpleAPI Network v1 33
6.4. Сообщения 34
6.5. Функции-утилиты для сетевого взаимодействия 35
7. Дополнительные функции-утилиты 36
8. Static Config 41
9. Аналоги `std::optional<T>`, `std::variant<T...>` и `std::visit()` 51

1. Введение

Библиотека SimpleAPI задумывалась как инструмент для удобной работы с файлами конфигураций с возможностью использования комментариев внутри файлов. А также передача конфигов по сети в указанном формате(**TODO**) (без комментариев) между программами.

То есть подразумевается несколько сценариев взаимодействия с конфигами:

- `программа-человек`: файлы с возможностью комментирования
- `программа-программа`: простая, с точки зрения программиста, упаковка значений в строку конфига, её передача до другого компьютера через сокет и простая распаковка в соотвествующие значения на другом компьютере; можно отправлять как JSON-объекты, так и простые потоки байтов - SimpleAPI Network сделает за программиста всё остальное.

Реализовано:

- полная поддержка формата JSON
- полная поддержка формата INI - стандарта не существует, но общие концепции сторонних реализаций сохраняются (свободно конвертируется в JSON и обратно)
- возможность указать стиль комментариев (символы границы 1-2 знака для однострочных и 1-3 знака - для многострочных), их чтение из файла и запись в него
- сетевое взаимодействие сервер-клиент в формате UDP с автоматической фрагментацией и дефрагментацией пакетов, перепосылкой недоставленных фрагментов (гарантированная стабильность только в локальной сети)

В планах:

- поддержка XML (начат поиск решения)

- поддержка YAML
- шифрование при отправке по сети

2. Зависимости

Минимальные требования к системе

ОС: `Linux`

Стандарт языка: `C++11`

Библиотеки

Проект написан собственными средствами стандарта языка C++ и зависит только от библиотеки `pthread` для класса SocketThread.

3.1. Интеграция с CMake

Минимальная версия CMake для сборки библиотеки 3.5.

Библиотеку можно добавить несколькими способами:

Как самостоятельный CMake проект

```
cd <SimpleAPI_dir>
mkdir -p build
cd build
cmake [-DCMAKE_BUILD_TYPE=Debug] .. # флаг -DCMAKE_BUILD_TYPE указывается
опционально
make -j$(nproc)
```

В директории build появится директория lib с собранной библиотекой в формате .so и директорией include со всеми заголовочными файлами библиотеки.

Как часть другого CMake проекта (add_subdirectory)

В составе вызывающего CMakeLists.txt следует указать следующий код CMake:

```
set(SIMPLE_API_MAIN_DIR ${CMAKE_CURRENT_LIST_DIR}/../..) # указать путь до
базовой директории библиотеки
include(${SIMPLE_API_MAIN_DIR}/utils.cmake) # для доступа к макросам библиотеки

# код для определения типа сборки
if(CMAKE_BUILD_TYPE STREQUAL "Release")
    set(SimpleAPI_lib_name SimpleAPI)
else()
    set(SimpleAPI_lib_name SimpleAPId)
endif()

add_subdirectory(${SIMPLE_API_MAIN_DIR}/SimpleAPI
${CMAKE_CURRENT_BINARY_DIR}/${SimpleAPI_lib_name})
```

```
# код для создания таргета основного приложения
add_executable(...)

# предполагается, что имя таргета совпадает с именем проекта (PROJECT_NAME)
# добавление зависимости от библиотеки для созданного таргета
target_link_libraries(${PROJECT_NAME} PUBLIC ${SimpleAPI_lib_name})
target_include_directories(${PROJECT_NAME} PUBLIC
${SIMPLE_API_MAIN_DIR}/SimpleAPI)
add_dependencies(${PROJECT_NAME} ${SimpleAPI_lib_name})
```

Через использование технологии find_package

Для использования этого варианта должна быть установлена библиотека `pkg-config`:

```
sudo apt-get install pkg-config # пример для Ubuntu
```

Пример кода CMake

```
set(SIMPLE_API_MAIN_DIR ${CMAKE_CURRENT_LIST_DIR}/../..) # указать путь до
базовой директории библиотеки
include(${SIMPLE_API_MAIN_DIR}/utils.cmake) # для доступа к макросам библиотеки
list(APPEND CMAKE_MODULE_PATH "${SIMPLE_API_MAIN_DIR}/cmake/find_package/") #
для поиска модулей Find*.cmake

# код для создания таргета основного приложения
add_executable(...)

# предполагается, что имя таргета совпадает с именем проекта (PROJECT_NAME)
if(CMAKE_BUILD_TYPE STREQUAL "Release")
    find_package(SimpleAPI REQUIRED)
    if(SimpleAPI_FOUND)
        target_link_libraries(${PROJECT_NAME} PUBLIC SimpleAPI)
        target_include_directories(${PROJECT_NAME} PUBLIC
${SimpleAPI_INCLUDE_DIRS})
    else()
        message(FATAL "SimpleAPI not found")
```

```
    endif(SimpleAPI_FOUND)
else()
    find_package(SimpleAPId REQUIRED)
    if(SimpleAPId_FOUND)
        target_link_libraries(${PROJECT_NAME} PUBLIC SimpleAPId)
        target_include_directories(${PROJECT_NAME} PUBLIC
${SimpleAPId_INCLUDE_DIRS})
    else()
        message(FATAL "SimpleAPId not found")
    endif(SimpleAPId_FOUND)
endif()
```

3.2. Интеграция с QMake

Ввиду ограниченности системы сборки QMake, реализовано только самое базовое включение SimpleAPI. Проект SimpleAPI не будет собираться как самостоятельная библиотека, будь то статичная или динамическая.

Для добавления в ваш QMake-проект SimpleAPI следует придерживаться такого шаблона:

(теоретический project.pro)

```
QT = core

CONFIG += c++11 cmdline

include(SimpleAPI_library/SimpleAPI.pri) # добавление SimpleAPI как источник
исходников

# продолжение описания исходников основного приложения
SOURCES += \
        main.cpp
```

3.3 Упаковка скомпилированной библиотеки в .deb пакет

После сборки CMake-проекта библиотеки пользователь может выполнть команду для архивации файлов библиотеки в самостоятельный пакет установки:

```
# переходим в папку билда библиотеки SimpleAPI
cd build
# вызов команды-упаковщика
make package
```

Настроенные пути установки пакета предполагают расположение библиотеки по пути `/opt/SimpleAPI`. Упакованный .deb пакет не имеет сторонних зависимостей.

4. Класс `simpleapi::Config`

Библиотека SimpleAPI предоставляет разработчикам прикладных программ удобный, а, главное, не перегруженный интерфейс в рамках своих инструментов. Базовым классом для работы со структурами настроек/конигураций является класс `simpleapi::Config`.

Класс `simpleapi::Config` представляет собой высокоуровневый интерфейс для хранения и управления примитивами значений. Таковыми на данный момент являются следующие типы: пустой объект (`nullptr`) `ValueType::eNull`, логический тип (`bool`) `ValueType::eBool`, числа (`long double`) `ValueType::eNumber`, буквы (`char`) `ValueType::eChar`, строковые значения (`std::string`) `ValueType::eString`, индексные массивы `ValueType::eArray`, и ассоциативные массивы `ValueType::eJson`. В перспективе будет добавлена поддержка собственных типов для хранения форматов XML и YAML (имеют расширенные правила, по сравнению с обычным ассоциативным массивом).

Числовые типы сейчас представлены типом `long double`, поэтому стоит учитывать его максимально допустимое хранимое значение в рамках используемой пользователем операционной системы. Для хранения очень больших чисел предлагается предварительно упаковывать их в std::string.

Пример работы:

```
#include "SimpleAPI.h" // единый заголовочный файл для использования любой из
возможностей библиотеки

int main(void) {
    using namespace simpleapi;

    CommentDesign cd;
    cd.with_comments = true; // выставляем флаг, если нужно искать комментарии
    Config cfg = ParseJson("/home/user/Downloads/my.json", cd); // аргумент cd
опциональный
    if(cfg.containsKey("key")) {
        // если в .toString() не указать аргументы, то значение выведется в
компактном виде (не зависит от типа)
        std::cout << "key exists, value: " << cfg["key"] << std::endl;
```

```
    } else {
        std::cout << "key not found" << std::endl;
    }

    return cfg.error() ? -1 : 0; // в случае ошибки парсинга будет заполнено
поле ошибки
}
```

Конструкторы `simpleapi::Config` позволяют создать объект `Config` на основе любого поддерживаемого классом типа. Примитивы создаются простой передачей, списки - через указанием первым аргументом требуемого типа. Ассоциативный массив предполагает указание типа и чередование ключей и значений:

```
Config cfg_int_1(100);
Config cfg_int_2(ValueType::eNumber, 100);
Config cfg_string_1("my_string");
Config cfg_string_2(ValueType::eString, "my_string");
Config cfg_bool(true);
Config cfg_bool(nullptr);
Config cfg_arr(ValueType::eArray, 15, "asd", "asd");
Config cfg_json(ValueType::eJson, "key1", 10, "key2", Config(ValueType::eArray,
1, 2, 34.5));
```

Используйте файл Config.h как источник актуальной информации о доступных методах. Обращайте внимание на отметки API_ в конце объявлений методов.

Класс simpleapi::Config предоставляет пользователю следующие возможности:

1. методы работы с комментариями (информацию об их специфике смотрите в соответсвтующем разделе)
2. методы работы с комментариями элементов внутри контейнеров (имя метода пишется через знак подчёркивания '_')
3. методы установки значения - используйте методы `.setValue()` или `operator=()`

4. методы получения значений примитивов: `.getBool()`, `.getNumber()` (есть методы для прямого преобразования чисел к нужному формату, например, `.getUInt32_t()`, `.get<T>()`), `.getString()`
5. методы работы с `Config` как контейнером (имя метода пишется через знак подчёркивания '_'), например, `.get_at()`, `.get_front()`, `.get_bool_at()` и т.д.
6. методы работы с контейнерами `Config` как набором итераторов: `.getRange()` и `.getMapRange()` для соответствующих типов контейнеров - индексные и ассоциативные
7. операторы сравнения одиночных объектов `Config` (опционально проверяются комментарии), а также сравнение контейнеров `Config` (опционально задаётся проверка сортировки элементов); опциональные параметры задаются при сравнении через метод `.isEqual()`
8. парсинг и запись `JSON` документов и файлов: `.toString()` с указанием формата вывода - `.toString(ConfigFormat::eJSON)`
9. парсинг и запись `INI` документов и файлов: `.toString()` с указанием формата вывода - `.toString(ConfigFormat::eINI)`
10. если парсинг завершился с ошибкой, то метод `.error()` вернёт сам факт ошибки, а метод `.getError()` даст строку с описанием ошибки; корректно прочитанные поля и значения не будут очищены
11. методы очистки значений `.clear()` и `.clearContainer()` (не удаляет сам контейнер, но очищает его содержимое)
12. методы добавления значений в контейнер: `.insert_*()` и `.push_*()`
13. методы удаления значений из контейнеров: `.erase_*()` и `.pop_*()`
14. методы получения константы значения с последующим удалением из контейнера: `.get_and_pop_*()`
15. методы получения информации о параметрах объекта: `.getType()`, `.is*()`, `.isEqual()` и т.п.
16. методы получения количества элементов внутри `Config`: контейнеры дадут актуальное количество элементов внутри, `eNull` всегда даст 0, другие типы - 1

17. методы определения наличия ключа или значения внутри контейнера: `.containsKey()` и `.containsValue()`
18. операторы присвоения (аналог `.setValue()`)
19. методы и функции работы с файлами для чтения и записи конфигов

Некоторые интересные факты о работе класса

Разрешено использовать ключи без наполнения - "".

Форматы `INI` и `JSON` свободно конвертируются один в другой. Из-за особенностей реализации формата INI бывают случаи, когда двойная конвертация даст другое расположение ключей (при записи в JSON создаются пустые ключи "", которые используются для хранения вложенных элементов).

Если объект `Config` является контейнером, то используя метод `.append()` можно добавить к нему элементы из другого контейнера того же типа.

Если ключа в `Config` ещё не существует, то он обращение к нему создаёт такой ключ и заполняет его `ValueType::eNull`.

Объект `Config` с типом `eNull` при использовании контейнерных методов группы добавлениия (`.insert*()`/`.push*` ()) самостоятельно преобразуется в указанный тип кконтейнера перед добавлением в него значения, переданного пользователем.

Класс `Config` не генрирует исключения (по понятным причинам это правило не касается `std::bad_alloc`). Любая ошибка работы класса меняет флаг `.error()` на `true` и заполняет `.getError()` строковым описанием ошибки.

Ошибка парсинга не затирает корректно прочитанные поля и значения, но при этом наличие ошибки всё ещё можно отследить методами, указанными выше.

Ввиду особенностей хранения больших дробных чисел, часть вводимой информации может оказаться избыточной. В конечном итоге в переменную числа запишется только некоторая часть дроби - знаки после фактически значимой части дроби пропадут уже при повторном чтении конфига. Например, число в текстовом конфиге `-0.40000000000000022204` будет корректно прочитано как `-0.4`. *(Пока нет отлаженного алгоритма, который бы позволил без потерь срезать избыточную информацию дроби)*

4.1. Формат JSON

Реализация, по большей части, соответствует стандарту RFC 8259.

В рамках SimpleAPI имеет тип `ValueType::eJson`.

ВАЖНО: автоматическая сортировка значений в SimpleAPI не предусмотрена.

Чтение конфига

Чтение организовано по принципу чтения по стандарту, с учётом поблажек на условности формата:

- возможно чтение нескольких типов значений: число (long double), логический флаг (true, false, +, -), строка, пустое значение (null), массив и вложенный Json. Любое другое значение будет воспринято как строка (если не нарушает синтаксис указания значения)
- ключ с пустым значением воспринимается как null-элемент
- кавычки нужны только если строка ключа или значения содержит несколько слов
- синтаксис "" - создаст конфиг с null-элементом для ключа ""
- перенос строки тоже считается разделителем, запятая необязательна
- в случае ошибки при чтении любой части документа успешно прочитанные значения не обнуляются, но заполняется описание ошибки
- метод Config::error() вернёт флаг наличия ошибки; текстовое описание ошибки доступно через вызов метода Config::getError()

Запись конфига

Запись Json происходит строго по стандарту (если не учитывать пользовательские комментарии - по умолчанию выключены) для совместимости с другими парсерами при передаче по сети.

Примеры кода

Примеры работы с Config как JSON-объектом рекомендуется изучать через написанные юнит-тесты `test_json.cpp` и `test_json_array.cpp`.

4.2. Формат INI

Формат INI не имеет принятого стандарта и реализуется всеми по-разному. SimpleAPI в этом плане старается брать лучшее из других реализаций.

Чтение и запись комментариев более подробно будут рассмотрены в следующих главах.

В рамках SimpleAPI так же имеет тип `ValueType::eJson`. Полностью совместим с форматом Json, но не рекомендуется записывать в файл массив Json (`ValueType::eArray`) в формате INI, т.к. обратное чтение даст в результате иную структуру документа.

Чтение конфига

Используется принцип "одна строка - одно значение".

Ключ может быть большой вложенности. Разделителем вложенностей ключа является слэш '/' и обратный слэш '\'.

Если переменная занимает несколько строк, то участвуют следующие правила проверки. Переменная не прочитана полностью, если выполняется хотя бы одно из правил:

- при чтении значения переменной не закрыта кавычка
- текущая строка заканчивается на знак обратного слэша '\'
- следующая строка начинается с пробела или табуляции

Каждое значение будет проверено на тип внутренним парсером.

По идее, в формате INI хранятся только строки, но SimpleAPI расширяет эту идею до способности самостоятельно решать, что это за тип данных, будь то число (long double), JSON, массив в формате JSON, bool, пустое значение (null) или обычная строка (всё то же, что и в формате JSON). При этом для строк наличие кавычек необязательно.

Пробелы в начале и конце значения при обработке будут проигнорированы.

Формат INI подразумевает, что на верхнем уровне документа может существовать несколько групп. Для примера, QSettings от фреймворка Qt начальной группе

назначает имя "MAIN". В SimpleAPI логика немного другая: элементы главной группы просто описываются до описания других групп. Таким образом, встреченные группы значений считаются подгруппами главного корня.

В итоге получим, например, такую структуру (формат JSON взят для наглядности): `{"a" = 1, "group" = {"b"="c", "d"="e"}}`. В QSettings эта же структура файла будет сохранена как `{"MAIN" = {"a" = 1}, "group" = {"b"="c", "d"="e"}}`.

Наличие названия группы полностью закрывает предыдущую группу. Наличие пустых строк или их отсутствие между группами не несёт логики.

Если в конфиге в рамках одной группы указано несколько одинаковых ключей, то их значения в совокупности являют собой массив с ключом в качестве имени массива.

Запись конфига

Логика записи итогового JSON-объекта в формате INI следующая:

- будут записаны по очереди ВСЕ одиночные значения
- массивы будут записаны по очереди с одним и тем же ключом
- JSON-объекты первого уровня образуют группы значений, в рамках которых все остальные внутренние контейнеры записываются как многосоставной ключ с разделителем имени через слэш '/'

Примеры кода

Примеры работы с Config как INI-документом рекомендуется изучать через написанные юнит-тесты `test_ini.cpp` и `test_ini_array.cpp`.

4.3. Формат XML (TODO)

4.4. Формат YAML (TODO)

5. Работа с комментариями

За работу с комментариями отвечают классы Comment и CommentDesign. В CommentDesign указываются правила для определения комментариев.

По умолчанию используется стиль комментариев C/C++, а то есть `// comment` для однострочных комментариев и `/* comment */` для многострочных комментариев.

Стиль написания комментариев определяет параметр CommentDesign корневого элемента.

За исключением форматов XML и YAML, библиотека позволяет указать стили для чтения и записи комментариев при работе с файлами. Форматы XML и YAML имеют заданный их стандартами стиль комментариев.

При парсинге заполняется CommentDesign::opt_multiline_column_size. Всегда.

В SimpleAPI работа с комментариями предполагает два вида комментариев:

1. комментарий ДО значения
2. комментарий ПОСЛЕ значения

Комментарий ДО значения

Несколько комментариев до значения будут восприняты парсерами как один многострочный комментарий.

Следует использовать методы Config::setPrefixComment() или более общий Config::setComment().

Комментарий ПОСЛЕ значения

Триггером начала отслеживания завершающего комментария является конец значения.

Триггером конца завершающего комментария считается перенос строки.

Ширина колонки многострочного комментария после значения не влияет на вывод (попытка записать в одну строку, если нет '\n').

Логика сравнения объектов CommentDesign

Следует держать в уме тот факт, что нельзя однозначно сравнить два CommentDesign с разными opt_multiline_column_size.

Проблема исходит из того, что при попытке записать такой стиль на комментарий, который как самую максимальную строку принимает, например N-1 (N - ширина колонки), то следующий парсер, который будет читать из этого результата в своём результате увидит ширину колонки как N-1.

Используемые библиотекой обработчики

За запись комментариев отвечает внутренняя функция tools::ToComment().

За парсинг отвечает функция tools::FromComment():

- однострочный комментарий:
  - может обрамляться 1 или 2 символами
  - если второй символ в строке пробел, то в *CommentDesign* запишется первый элемент и 0
- многострочный комментарий:
  - может обрамляться 1 или 2 символами (3 символ указывается как замена 1 при закрытии блока комментариев)
  - правильную границу комментария определяет функция чтения конфига
  - здесь проверка на существование границы в конце будет опускаться

Однострочные комментарии (пример)

Пример добавления нового шаблона поиска:

```
CommentDesign cd;
cd.oneline_comment_variants.push_back({'#', 0});
```

Вектор принимает `std::array<char, 2>`. Оба символа являют собой строку, которая обозначает начало однострочного комментария и действует до переноса строки. Если второй символ указан как число 0 (без кавычек), то начало комментария определяется строго одним символом.

Многострочные комментарии (пример)

Пример добавления нового шаблона поиска:

```
CommentDesign cd;
cd.multiline_comment_variants.push_back({'{', '|', '}'});
```

Вектор принимает `std::array<char, 3>`. Суть описания похожа на однострочные комментарии за тем исключением, что наличие третьего символа обозначает замыкающую границу комментария.

Примеры использования:

- `{'{', '|', '}'}` -> `{| multiline comment |}`
- `{'{', 0, '}'}` -> `{ multiline comment }`
- `{'{', '|', 0}` -> `{| multiline comment |{`

Важные замечания

with_comments:

Комментарии не будут прочитаны или записаны, если поле `CommentDesign::with_comments` в составе объекта CommentDesign не равно true.

CommentDesign

Переменная объекта CommentDesign в составе Config будет обновлена и дополнена при чтении нового файла/входной строки конфига.

Колонка

Для многострочных комментариев можно ограничить ширину колонки. Строки будут обрезаны автоматически.

Для включения функции следует использовать поле `CommentDesign::opt_multiline_column_size`.

Значение 0 отключает параметр.

Рамка

Для многострочных комментариев можно указать стиль рамки, например:

```
CommentDesign cd;
cd.opt_multiline_border = '*';
```

Выведет так:

```
/**********
* comment *
**********/
```

Значение 0 отключает параметр.

Стиль многострочного комментария

Для многострочных комментариев без рамки вывод по умолчанию указывает начало и конец комментария на отдельных строках, например:

```
/*
multiline
comment
*/
```

Поле `CommentDesign::opt_multiline_border_at_content_line=true` изменяет этот момент:

```
/* multiline
   comment   */
```

Итоговый вид комментариев в строке вывода

Комментарий можно указать ДО значения и ПОСЛЕ него.

Несколько комментариев ДО значения сформируют один большой комментарий.

Комментарий ПОСЛЕ значения применяется только в том случае, если после написания значения комментарий был начат до переноса строки.

Комментарий ПОСЛЕ значения может быть только одним:

```
{
        // много
        // комментариев
        // ДО значения
        key="value", // комментарий ПОСЛЕ значения
        /* ещё
        один пример */
        /* вторая часть второго примера */
        key2=12465 /* большой
                комментарий
                после значения
                */
}
```

Уникальная обработка колонки комментария ПОСЛЕ значения

При записи комментария ПОСЛЕ значения параметр `CommentDesign::opt_multiline_column_size` не влияет на ширину колонки, если в комментарии нет хотя бы одного явного переноса строки (\n). В таком случае будет сделана попытка записать комментарий как однострочный (для лучшей читабельности итогового конфига).

6. Сетевое взаимодействие

Библиотека SimpleAPI предоставляет возможности по относительно простому добавлению взаимодействия между удалёнными программами посредством сокетов.

Вариантов несколько:

Ручное управление

Использовать удобную обёртку над сокетами - UdpSocket и работать через методы Socket::sendRawMsg() и Socket::recvRawMsg().

SimpleAPI Network

Предоставить библиотеке право самой управлять низкоуровневой логикой.

Пользователю нужно будет только вызывать метод отправки и настроить callback, который библиотека будет вызывать при получении нового сообщения.

6.1. Сокеты

Сокеты в SimpleAPI представлены классом `Socket`, в конструкторе которого нужно указать его тип `SocketType`. На данный момент это типы `SocketType::eUDP` и `SocketType::eTCP` (*TODO*).

Соответствуя логике SimpleAPI, пользователь не обязан знать детали внутренней реализации тех или иных классов. Поэтому абстрактный класс `Socket` имеет двух потомков: `UDPSocket` и `TCPSocket` (*TODO*).

Детально настроить логику внутреннего поведения функций внутри этих классов можно с помощью вспомогательного класса `SocketSettings`. SocketSettings даёт возможность выставить и получить информацию о следующих параметрах:

- включить и выключить шифрование на сокетном канале (*TODO*), `void enableChiphering(bool enabled = false) noexcept`
- проверить активность шифрования на сокетном канале, `bool isChipheringEnabled() noexcept`
- выставить таймер неактивности, `void setInactivityTimer(long milliseconds = 10000) noexcept`
- выбрать уровень проверки целостности выкл/8B/16B/32B, `void setCrcLevel(CRC crc_level = CRC::eCRC_OFF) noexcept`
- указание использовать указанную версию внутреннего сетевого протокола SimpleAPI как максимальную, `void setApiVersion(ApiVersion version = GetLastApiVersion()) noexcept`

Автоматизированный вариант сокетов будет представлен в следующем разделе.

Вспомогательный класс `IpPort` используется для удобного хранения и передачи пары `IP-адрес - порт` при использовании SimpleAPI. Также этот класс умеет применять значение на основе строки формата `X.X.X.X:port`, в случае неуспеха вернёт false.

UDP сокеты

Пример создания сокета:

```
#include "SimpleAPI.h"

int main()
{
    using namespace simpleapi;

    // инициализация сокета в ручном режиме
    IpPort local_ip_port{"127.0.0.1", 12345};
    SocketSettings settings;
    UDPSocket sock(local_ip_port, settings);

    // отправка сообщения до адресата
    std::string message = "Hello world!";
    Packet packet = message.data();
    IpPort remote_ip_port{"192.168.0.100", 54321};
    sock.sendRawMsg(remote_ip_port.ip, remote_ip_port.port, packet);

    // ожидание ответа от адресата в течение пяти секунд
    PacketMessage answer = sock.recvRawMsg(5000);
    // проверяем, что первый пришедший на сокет пакет оказался от адресата
remote_ip_port
    if (answer.m_ip_port == remote_ip_port && !answer.m_packet.empty())
    {
        std::cout << "OK" << std::endl;
    }
    else
    {
        std::cout << "FAIL" << std::endl;
    }
}
```

TCP сокеты

TODO: на данный момент нет реализации

6.2. Поток для автоматизации работы сокетов

Класс `SocketThread` предназначен для вынесения всех сокетных соединений в отдельный поток с автоматизацией работы с сокетами.

Главное преимущество этого класса заключается в том, что достаточно один раз объявить добавление к объекту `SocketThread` нового экземпляра сокета и дальнейшее взаимодействие с сокетами будет происходить по логике функций обратного вызова (callback).

Улучшения по сравнению с ручными сокетами

- автоматическая фрагментация пакетов при отправке и их дефрагментация при получении
- если полученный пакет без ошибок сконвертировался в Json, то будет вызван callback для полученного Json-пакета
- автоматическая проверка наличия соединения (пинги раз в несколько секунд)
- уведомления о новых соединениях, входящих пакетах/Json-документах и разрывах соединений при долгом отсутствии активности адресата

Основные параметры автоматизированных сокетов

При использовании такого подхода, `SocketSettings` позволяет настроить ещё и такие параметры:

- указать размер одного фрагмента, `void setMaxLength(uint16_t max_length = 1500) noexcept`
- указать количество отправляемых фрагментов за один проход цикла, `void setMaxMsgsSentOnTick(int max_msgs_sent_on_tick = -1/*все разом*/) noexcept`
- указать функцию для callback-вызова при приходе нового пакета, `void setRecvPacketCallback(RecvPacketMessageCallback callback = nullptr) noexcept`

- указать функцию для callback-вызова при приходе нового JSON-пакета, `void setRecvJsonCallback(RecvJsonMessageCallback callback = nullptr) noexcept`
- указать функцию для callback-вызова при создании нового входящего соединения, `void setNewConnectionCallback(ConnectionCallback callback = nullptr) noexcept`
- указать функцию для callback-вызова при дисконнекте соединения (провал проверки соединения), `void setConnectionResetCallback(ConnectionCallback callback = nullptr) noexcept`

Логирование автоматизированных сокетов

Так как `SocketThread` является классом-наследником от `LoggerSetting`, то следует здесь же объяснить уже его возможности:

- включить вывод логов, `void enablePrintLogLevel(bool enabled = false) noexcept`
- включить вывод времени лога при выводе, `void enableLogTime(bool enabled = false) noexcept`
- включить привязку колонок при выводе к правому краю, `void enableNameColumnRightAlign(bool enabled = false) noexcept`
- указать стандартную ширину заголовка колонки при выводе, `void setNameColumnSize(int size = -1) noexcept`
- указать уровень детализации логов, `void setLogLevel(logs::LEVEL level = logs::eINFO) noexcept`
- указать функцию для callback-вызова для вывода логов, `void setLogCallback(LogCallback callback = nullptr) noexcept`
- указать функцию для callback-вызова для вывода логов ошибок, `void setLogErrorCallback(LogCallback callback = nullptr) noexcept`
- указать функцию для callback-вызова для вывода цветных логов, `void setColorLogCallback(LogCallback callback = nullptr) noexcept`
- указать функцию для callback-вызова для вывода цветных логов ошибок, `void`

`setColorLogErrorCallback(LogCallback callback = nullptr) noexcept`

Безопасность потоков

Обратите внимание, что все callback-функции будут вызваны из потока SocketThread. Вся потоконезависимая часть программы реализуется самим пользователем библиотеки.

Пример кода использования SocketThread (в минимальном виде):

```
#include "SimpleAPI.h"

// код обработчика нажатия Ctrl+C (SIGINT)
bool isRunning = true;
void signalHandler(int signal) {
    if(signal == SIGINT) {
        std::cout << "\n";
        isRunning = false;
    }
}

int main()
{
    signal(SIGINT, signalHandler); // включить обработчик для SIGINT

    using namespace simpleapi;

    // создание callback-функции на входящий Json
    auto RecvJson = [](JsonMessage jm) -> void {
        std::cout << logs::get_time_string() << " "
                  << logs::columned(MAIN_COLOR, "[SERVER]",
                                    NAME_COLUMN_SIZE,
                                    NAME_COLUMN_RIGHT_ALIGN) << " "
                  << logs::to_color_string({logs::COLOR::eGREEN_BG,
logs::COLOR::eWHITE_FG},
                                           "recv json: " + jm.toString())
```

```
                << std::endl;
    }

    // создание callback-функции на входящий пакет в виде байтов
    auto RecvData = [](PacketMessage pm) -> void {
    std::cout << logs::get_time_string() << " "
              << logs::columned(MAIN_COLOR, "[SERVER]",
                                NAME_COLUMN_SIZE,
                                NAME_COLUMN_RIGHT_ALIGN) << " "
              << "recv data: 0x" << utils::ToHexString(pm.m_packet)
              << std::endl;
    };

    IpPort local_server{"127.0.0.15", 31115};
    SocketSettings settings;
    settings.setRecvPacketCallback(RecvData);
    settings.setRecvJsonCallback(RecvJson);

    // создание потока сокетов
    SocketThread st;

    // добавление к потоку нового UDP-сокета
    st.addSocket(eUDP, server, settings);

    // запуск добавленного потока
    st.startThread();

    IpPort remote_ip_port{"127.0.0.16", 51113};
    Config json_message;
    json_message["Hello"] = "World!";

    bool isNeedSend = true; // флаг для одноразовой отправки
    while(1)
    {
        // флаг вызовется при нажатии Ctrl+C
        if(!isRunning) {
            st.stopThread();
            break;
```

```
        }

        if(isNeedSend)
            st.send(server, remote_ip_port, json_message);
            isNeedSend = false;
        }

        usleep(100);
    }
}
```

6.3. Описание структуры пакетов SimpleAPI Network v1

Новое подключение автоматически добавляется в отслеживаемые и пингуемые до тех пор, пока не пройдёт указанное количество секунд в таймере SockerSettings.setInactivityTimer(N), где N по умолчанию равно 10 с.

Отправляемое сообщение делится на фрагменты меньшей длины согласно настройкам из SocketThread.

В заголовке каждого сообщения указывается следующая информация:

- тип пакета, Control(0) или Data(1) (1 бит)
- версия используемого Network API (2 бита)
- флаг первого сегмента сообщения (1 бит)
- флаг последнего сегмента сообщения (1 бит)
- флаг включения шифрования (1 бит)
- тип проверки контрольной суммы (2 бита)
- глобальный счётчик пакетов (сквозной) (8 бит)
- локальный счётчик пакетов (сквозной) (8 бит)
- (при наличии, только для первого фрагмента сообщения) контрольная сумма (зависит от типа: 0/8/16/32 бит(ов))
- размер полного сообщения (16 бит)
- передаваемые данные (по заполнению)

Контрольная сумма считываается для всего сообщения уже после сборки пакета.

6.4. Сообщения

Сокеты в SimpleAPI Network в ответном пакете возвращают объект `PacketMessage`.

В случае, если сообщение является JSON-документом, то перед вызовом callback-функции `PacketMessage` будет преобразован в `JsonMessage`.

Класс PacketMessage

Доступные поля:

- `m_ip_port` - адрес и порт отправителя
- `m_packet` - полученные данные

Класс JsonMessage

Доступные поля:

- `m_ip_port` - адрес и порт отправителя
- `m_json` - полученные данные в формате Json, если исходный PacketMessage можно было преобразовать в Json

6.5. Функции-утилиты для сетевого взаимодействия

7. Дополнительные функции-утилиты

В ходе разработки библиотеки было сформировано множество полезных функций, которые могут показаться полезными пользователю. Данная статья рассказывает о некоторых из них.

Пространство имён: `simpleapi::utils`.

1. CharInString

Сигнатура:

`bool CharInString(const char ch, std::string symbols) noexcept`

Суть:

Определяет, есть ли указанный символ ch среди строки symbols.

2. ToHexString

Сигнатура:

`std::string ToHexString(const std::vector<uint8_t>& data) noexcept`

Суть:

Преобразует вектор байтов в строку формата HEX.

3. FromHexString

Сигнатура:

`std::vector<uint8_t> FromHexString(std::string str) noexcept`

Суть:

Преобразует строку формата HEX в вектор байтов.

4. GetStringCharCount

Сигнатура:

`size_t GetStringCharCount(const std::string &str, bool only_visible)`

Суть:

Указывает количество символов в строке формата UTF-8. Опционально можно узнать количество видимых символов.

5. GetNormalizeString

Сигнатура:

`std::string GetNormalizeString(const std::string &input) noexcept`

Суть:

Соберёт новую строку без пробелов и переносов строк.

6. RemoveIllegalSpaces*

Сигнатуры:

`void RemoveFrontIllegalSpaces(std::string& string) noexcept` `void RemoveEndIllegalSpaces(std::string& string) noexcept` `void RemoveIllegalSpaces(std::string& string) noexcept`

Суть:

Удаляют пробелы и табуляции в начале/конце строки.

7. SplitWithoutColumned

Сигнатура:

`SplittedLines SplitWithoutColumned(const std::string& input_string) noexcept`

Суть:

Обрежет входную строку на список строк (SplittedLines). Учитываются только пользовательские переносы строк.

8. SplitToColumns

Сигнатура:

`SplittedLines SplitToColumns(const std::string& input_string, const size_t column_size) noexcept`

Суть:

Обрежет строку на подстроки с заданной шириной.

Если хотя бы одна строка неделима и превышает предел, то остальные строки будут выровнены по новому пределу.

Многоточие считается частью слова и не переносится на другую строку.

Пользовательские переносы строк будут сохранены.

9. VStringToString

Сигнатура:

`std::string VStringToString(const VString& input_vec, const bool need_quotes = false) noexcept`

Суть:

Преобразует вектор строк в одну строку, при этом изначальные строки будут соединены знаком переноса '\n'.

10. SplitString

Сигнатура:

`SplittedLines SplitString(const std::string input_string, char split_char, bool with_empty_strings = true) noexcept`

Суть:

Разделяет строку на подстроки с указанным символом-разделителем.

11. GetAllFilesByMask

Сигнатура:

`std::vector<std::string> GetAllFilesByMask(const std::string& path_to_dir, const std::string& regex, const int& max_level = 0) noexcept`

Суть:

Выдаёт список всех файлов в указанной директории с возможностью указать regex-маску поиска. Выполняет рекурсивный поиск всех путей до файлов по указанной маске, начиная с указанной директории до глубины N (-1 - бесконечная глубина поиска).

12. ToString()

Сигнатура:

`std::string ToString(const T& value) noexcept`

Суть:

Перевод чисел, bool, внутренних типов библиотеки в строковое представление.

13. CreateBoolFromString()

Сигнатура:

`bool CreateBoolFromString(const std::string& input);`

Суть:

Создаст значение bool из строкового представления.

14. CreateLLongFromString()

Сигнатура:

`long long CreateLLongFromString(const std::string& input);`

Суть:

Создаст значение long long из строкового представления.

15. CreateLDoubleFromString()

Сигнатура:

```
long double CreateLDoubleFromString(const std::string& input);
```

Суть:

Создаст значение long double из строкового представления.

8. Static Config

Об идее

В ходе разработки и тестирования возможностей библиотеки SimpleAPI была сформирована идея, что класс Config позволяет многое, но упускает иногда главное для программиста, работающего с файлами конфигураций. А именно - подсветку доступных переменных при обращении к объекту Config внутри IDE.

Проблема исходит из того, что объект Config формируется на ходу, то есть ему совершенно неважно, что вы дадите ему на вход.

В связи с этим появилась концепция, что программист-пользователь библиотеки может через макросы описать необходимые ему типы и названия полей структуры, а библиотека берёт на себя её заполнение и валидацию средствами класса Config и паттерн X-macro.

На данный момент эта технология поддерживает:

- одиночные примитивы (строки, числа, bool, char)
    - `simpleapi::Config` хранит числовые значения в формате long double, поэтому объявление в структуре поля float с последующей записью в Config, а затем Config в файл, - запишет в файле мусорный конец значения; обратное чтение восстановит изначальный float
- вложенные структуры, описанные через эту же технологию
- перечисления (enum) - при этом в конечном файле конфига хранится строковое представление поля enum
- STL-контейнеры:
    - `std::vector<T>`
    - `std::list<T>`
    - `std::forward_list<T>`
    - `std::deque<T>`

    - `std::set<T>`
    - `std::multiset<T>`
    - `std::unordered_set<T>`
    - `std::unordered_multiset<T>`
    - `std::queue<T>`
    - `std::priority_queue<T>`
    - `std::stack<T>`
    - `std::array<T, size>`
    - `std::bitset<size>`
- ассоциативные STL-контейнеры (K = строка, число или bool):
    - `std::map<K, T>`
    - `std::multimap<K, T>`
    - `std::unordered_map<K, T>`
    - `std::unordered_multimap<K, T>`
- `std::pair<K, T>`
- динамический ассоциативный массив `simpleapi::Config`
- опциональное значение `simpleapi::Optional<T>`
    - запишет в файл конфига две переменные: `<key>` и `<key>_optional`
- "широкие" строки std::wstring использовать разрешено, но строго для формата UTF-8 (требование SimpleAPI Config)
- вариативный тип `simpleapi::Variant<T...>` (аналог `std::variant<T...>` из C++17)
    - комментарии и валидаторы данного типа применятся ко всем типам одинаково <T...> (нельзя задать уникальные правила)

Макросы описания

X-Macro

Макросы этой группы целиком описываются пользователем, но суть в следующем. Каждый из описанных ниже макросов принимает на вход определённое количество параметров. Не все из них обязательные. Это как с вызовом функций, у которых переменное количество параметров.

SAPI_REGISTER_ENUM(EnumName, X_MACRO_FOR_CURRENT_ENUM_FIELDS, UnderlyingType)

Параметры основного макроса:

- `EnumName`
  - итоговое имя класса перечисления (enum class)
- `X_MACRO_FOR_CURRENT_ENUM_FIELDS`
  - макрос X-Macro, объявленный пользователем ранее, описывающий список значений (формат `(X)`)
- `UnderlyingType`
  - (опциональный) базовый класс, от которого будет наследован enum class

Параметры X-Macro:

- `element`
  - имя поля enum class
- `value`
  - (опциональный) числовой параметр указанного значения

Вспомогательные функции, которые создаёт основной макрос регистрации перечисления:

- `std::string ToString(value)`

    - функция для перевода конкретного значения внутри enum class к строковому виду
- `void FromString(input_str, out_value)`
    - функция для преобразования строки в значение enum class; если значение не будет распознано - применится поле :: *UNDEFINED_STATE*
- `std::string EnumPossibleVariants(value)`
    - функция, которая выводит в строку все возможные валидные поля данного enum class

SAPI_REGISTER_CONFIG(StructName, X_MACRO_FOR_CURRENT_STRUCT_FIELDS)

Параметры основного макроса:

- `StructName`
    - итоговое имя структуры
- `X_MACRO_FOR_CURRENT_STRUCT_FIELDS`
    - макрос X-Macro, объявленный пользователем ранее, описывающий список параметров (формат `(X, Y)`)

Параметры X-Macro:

- `type`
    - тип переменной, которая будет описана внутри структуры
- `name`
    - имя переменной; будет указано и внутри структуры, и в итоговом файле Config
- `default_value`
    - значение по умолчанию, будет применено ещё до вызова .loadConfig()
- `lambda`

    - (опциональный) лямбда-функция, которая принимает на вход аргумент `const type& val`, описывающий тип переменной; значение будет применено к прочитанному библиотекой значением из файла конфига
- `prefix_comment`
    - (опциональный) строка комментария, которая будет выведена в файле конфига на строке перед значением
- `suffix_comment`
    - (опциональный) строка комментария, которая будет выведена в файле конфига на строке значения сразу после него

ВАЖНО: если поле внутри себя должно иметь запятые, то нужно использовать либо `using XXX = ...;`, либо скобки `(...)`

Вспомогательные функции, которые создаёт основной макрос регистрации структуры:

- `bool loadConfig(const simpleapi::Config& config)`
    - заполнит структуру значениями из config; вернёт true если все поля были прочитаны корректно и провалидировано лямбдой (при наличии)
    - в случае ошибки загрузки значения заполнит глобальную строку `simpleapi::static_config_error_str`
- `simpleapi::Config saveConfig() const`
    - сохранит значения полей структуры в объект Config

Примеры использования

Объявление enum class

```
// объявление X-Mac
#define MY_ENUM_FIELDS(X) \
    X(eVal_1)             \
    X(eVal_2)
```

```
// создаст enum class { eVal_1, eVal_2, _UNDEFINED_STATE_ };
SAPI_REGISTER_ENUM(MyEnum, MY_ENUM_FIELDS)

// создаст enum class : uint8_t { eVal_1, eVal_2, _UNDEFINED_STATE_ };
SAPI_REGISTER_ENUM(MyEnum2, MY_ENUM_FIELDS, uint8_t)
```

Объявление struct

```
// объявление X-Macro (X - поля с поддержкой работы с SimpleAPI, Y - поля без
работы с SimpleAPI)
// поле int_value описывает только обязательные параметры
// поле str_value имеет функцию валидации, которая проверяет минимальную длину
прочитанной строки
// поле int_vec не имеет валидации, но имеет комментарий перед значением
#define MY_STRUCT_FIELDS(X, Y)                                              \
    X(int,              int_value, 0)                                       \
    X(std::string,      str_value, "bla-bla-bla",                           \
        [](const std::string& s) -> bool { return s.size() > 10; }          \
    X(std::vector<int>, int_vec,   {}, nullptr, "is vector of integers") \
    Y(char,             ch,        'a')

// создаст struct MyStaticConfig с полями, описанными выше
SAPI_REGISTER_CONFIG(MyStaticConfig, MY_STRUCT_FIELDS)

// описание структуры, которая внутри содержит внутреннюю структуру,
объявленную ранее
#define MY_STRUCT2_FIELDS(X, Y)                     \
    X(char,           ch,           'a')            \
    X(MyStaticConfig, inner_struct, MyStaticConfig{})

SAPI_REGISTER_CONFIG(MyStaticConfig2, MY_STRUCT2_FIELDS)
```

Применение в коде

Для примера берём структуру из предыдущего кода:

```
MyStaticConfig2 static_config;
```

```
if(!static_config.loadConfig())
{
    std::cerr << "Error: incorrect config!" << std::endl;
}
```

Пример использования внешней переменной в лямбде

Стандарт C++11 не позволяет передавать владение глобальным объектом внутрь лямбды. Зато позволяет её явное использование.

```
#include <...> // включение необходимых библиотек на усмотрение разработчика

std::string my_global_string;

// объявляем Static Config структуру
#define FIELDS(X, Y)                                                    \
    X(std::vector<int>, vec, std::vector<int>,                          \
        [](const std::vector<int>& v) -> bool {                         \
            bool b = v.size() >= 10;                                    \
            if(b) {                                                     \
                return true; /*успех валидации*/                        \
            }                                                           \
            /* заполняем строку ошибки */                              \
            my_global_string = "В поле \"vec\" слишком мало элементов!" \
            return false; /* выходим с ошибкой валидации */             \
        } )

SAPI_REGISTER_CONFIG(MyStruct, FIELDS)

int main() {
    using namespace simpleapi;
    // считываем конфигурацию
    Config cfg = ReadIniFile("path/to/my/conf.ini");

    // заполняем статическую структуру на основе динамической
    MyStruct ms;
    if(!ms.loadConfig(cfg))
    {
```

```
        // при загрузке произошла ошибка валидации, выводим наше сообщение
        std::cout << "MyStruct loading error.";
        if(!my_global_string.empty())
            std::cout << " " << my_global_string;
        std::cout << std::endl;
        return 1;
    }

    return 0;
}
```

Пример использования simpleapi::Variant

```
#include <...> // включение необходимых библиотек на усмотрение разработчика

// объявляем Static Config структуру
using MyVariant = simpleapi::Variant<int, std::string, std::nullptr_t>;
MyVariant var;
//-----------------------------------------------------------------
// обычная лямбда
#define FIELDS(X, Y)                                                    \
    X(MyVariant, variant, MyVariant(15), [](const MyVariant& var) -> bool { \
        return Visit(var                                                \
                    , [](const int& i)           -> bool { return true; }  \
                    , [](const std::string& str) -> bool { return true; }  \
                    , [](std::nullptr_t)         -> bool { return true; }  \
        );                                                              \
    })
SAPI_REGISTER_CONFIG(MyConfig, FIELDS1)

// лямбда с именем поля
#define FIELDS_2(X, Y) \
X(MyVariant, variant, MyVariant(15), [](const MyVariant& var, const
std::string& key_name) -> bool { \
        return Visit(var \
        , [&key_name](const int& i)           -> bool { std::cout << "key: " <<
key_name << std::endl; return true; } \
```

```
        , [&key_name](const std::string& str) -> bool { std::cout << "key: " <<
key_name << std::endl; return true; } \
        , [&key_name](std::nullptr_t)         -> bool { std::cout << "key: " <<
key_name << std::endl; return true; } \
        ); \
    })

SAPI_REGISTER_CONFIG(MyConfig_2, FIELDS_2)
//-----------------------------------------------------------------

int main() {
    using namespace simpleapi;

    MyConfig mcfg;
    Config cfg;

    auto run = [&](std::string&& input) {
        cfg.parseJson(input);
        mcfg.loadConfig(cfg);
        Visit(mcfg.variant
            , [](const int& i)           { std::cout << "int: " <<
std::to_string(i) << std::endl; }
            , [](const std::string& str) { std::cout << "string: " << str <<
std::endl;            }
            , [](std::nullptr_t)         { std::cout << "std::nullptr_t" <<
std::endl;             }
            );
    };

    run("{variant:\"asd\"}");
    run("{variant:100}");
    run("{variant:null}");

    //------------------------
    MyConfig_2 mcfg_2;

    auto run = [&](std::string&& input) {
        cfg.parseJson(input);
```

```
        mcfg_2.loadConfig(cfg);
        Visit(mcfg_2.variant
            , [](const int& i)           { std::cout << "int: " <<
std::to_string(i) << std::endl; }
            , [](const std::string& str) { std::cout << "string: " << str <<
std::endl;            }
            , [](std::nullptr_t)         { std::cout << "std::nullptr_t" <<
std::endl;             }
            );
    };

    run("{variant:\"asd\"}");
    run("{variant:100}");
    run("{variant:null}");

    return 0;
}
```

9. Аналоги `std::optional<T>`, `std::variant<T...>` и `std::visit()`

Стандарт C++17 предоставил опциональные значения через `std::optional<T>`, а также инструментарий для выполнения кода над переменной в зависимости от её текущего типа, при этом тип может оказаться любым из указанных на этапе компиляции, — это класс `std::variant<T...>` и функция `std::visit()`.

Библиотека SimpleAPI привносит эти удобные механизмы в кодовую базу на основе C++11. Они представлены элементами `simpleapi::Optional<T>`, `simpleapi::Variant<T...>` и `simpleapi::Visit()` соответственно.

Класс simpleapi::Optional<T>

Шаблонный класс `simpleapi::Optional<T>` даёт разработчику возможность гибкой настройки, внезапно, опциональных параметров внутри создаваемых структур. Реализован весь интуитивный функционал инструмента: присваивание значения включает параметр и заполняет его внутреннее значение, метод `.set(value)` делает ровно то же самое, вызов `.value()` даёт возможность использовать внутреннее значение переменной, использование опциональной переменной внутри условия `if` неявно приводится к `bool` (аналог вызова `.isValid()`). Однако основным отличием реализации `simpleapi::Optional<T>` от стандартного `std::optional<T>` является то, что метод `.unset()` сбрасывает только флаг использования значения, но не само значение. Этот механизм позволяет создавать невероятно гибкие системы, вроде передачи переменной внутрь функции, внутри которой значение будет воспринято как невалидное и отключено, внешний код может заполнить лог ошибки вроде такого: "Переменная X находится вне диапазона, прочитанное значение 15!"

Класс simpleapi::Variant<T...>

Данный класс представляет собой мультикласс, внутри которого может находиться значение любого из указанных при объявлении типов. Он чем-то напоминает структуру union, но при этом во время выполнения кода можно узнать какого типа значение сейчас находится в объекте этого класса.

`simpleapi::Variant<T...>` имеет механизмы расчёта выравнивания по максимальному значению выравнивания памяти из всего списка типов `T...`. Также объект такого

класса всегда занимает объём памяти, необходимый самому тяжёлому из подтипов. Если тип `T` обращается к куче самостоятельно (например, `std::string`), то `simpleapi::Variant<T...>` никак эти механизмы не изменяет.

Реализованы методы для получения значения для определённого подтипа `.get<T>()`, но пользователь библиотеки должен сам следить за тем, что данный конкретный тип `T` сейчас действительно является текущим для `simpleapi::Variant<T...>`. В случае, если вызываемый тип не совпадает - получаем UB (Undefined Behavior), т.к. вызов происходит обычным `reinterpret_cast<T>` в целях экономии процессорного времени.

Дополнительно реализованы методы с проверкой забираемого типа - в случае, если тип заполнен, то метод `.get_if<T>()` вернёт указатель на данные типа `T`.

Инициализация по умолчанию объекта `simpleapi::Variant<T...>` подразумевает вызов конструктора по умолчанию для типа, находящегося первым в списке `T...`. Если не передавать в конструктор `simpleapi::Variant<T...>` сразу объект нужного типа, то компилятор сам передаст аргументы для вызова конструктора соответствующего типа.

Класс `simpleapi::Variant<T...>` имеет механизм защиты от перезаписи значения для типа, который имеет приписку `const` при описании списка типов `T...`. Однако типы `const char*` и `const wchar*` имеет исключение из данного правила - константность указателя (`const char*`) запрещает менять данные по указателю, но не запрещает изменить сам указатель.

Присваивание объекту `Variant` другого `Variant` всегда перезаписывает даже типы с припиской `const`, независимо от общности списка типов внутри.

Предусмотрен метод `.emplace(args...)` для вызова конструктора внутреннего объекта `simpleapi::Variant<T...>` пользователем напрямую, без лишних аллокаций и перемещений аргументов.

Присвоение значения того же типа переменной `simpleapi::Variant<T...>` не перевыделяет память для объекта `T`, а сразу присваивает внутреннему полю новое значение.

Класс `simpleapi::Variant<T...>` имеет поддержку в подсистеме SimpleAPI Static Config. Подтипами класса могут быть любые из поддерживаемых Static Config типов.

Следует также учитывать, что `operator==()` проверяет только фактическое хранимое значение внутри `simpleapi::Variant<T...>`, он не проверяет совместимость списков

T....

Функция simpleapi::Visit()

Функция simpleapi::Visit() является функцией-спутником simpleapi::Variant<T...> и повторяет поведение своего аналога из стандарта C++17, но имеет свои отличия. Например, реализация SimpleAPI не предусматривает работу с аргументом overloaded. То есть нельзя передать структуру с перегруженными operator() под каждый из типов simpleapi::Variant<T...>.

Вызов функции имеет следующий вид:

```
simpleapi::Visit(variant_object, args...);
```

, где args... представляет собой независимый друг от друга список лямбд, указателей на лямбды, указателей на свободные функции и т.д.

Можно передать сколь угодно большой список аргументов - библиотека это предусматривает, но обязаны быть предоставлены реализации под каждый из подтипов simpleapi::Variant<T...>. В ином случае библиотека сообщит об ошибке времени компиляции. Порядок описания функций никак не зависит от порядка указания подтипов simpleapi::Variant<T...>.

Также список функций/лямбд обязан иметь один и тот же возвращаемый тип. Если пользователь забывает об этом правиле, библиотека напомнит об этом ошибкой компиляции с описанием проблемы.

Пример использования

```
#include <iostream>
#include "SimpleAPI.h"

bool func_int(const int& i)
{
    std::cout << "is integer: " << std::to_string(i) << std::endl;
    return i > 0;
}
```

```
int main()
{
    using namespace simpleapi;
    auto lambda_nullptr = [](std::nullptr_t) -> bool
    {
        std::cout << "is null" << std::endl;
        return false;
    };

    Variant<int, std::nullptr_t, bool> var(false);
    bool result = Visit(var,
                        func_int,
                        lambda_nullptr,
                        [](bool b) -> bool
                        {
                            std::cout << "is bool: " << (b ? "true" : "false")
<< std::endl;
                            return b;
                        }
                        );

    return result;
}
```

При выполнении данного кода получим строку `is bool: false`, причём переменная *result* будет содержать значение `false`.